Calico Launcher

1. Product Overview and Vision

- A full featured emulation frontend inspired by the 3DS UI built for single and dual screen Android devices
    - Mainly focused for dual screen
- Games can be sorted and collected
- Android apps won't be in the main emulation section, but rather in a customizable bottom navigation bar similar to the Windows taskbar
- Backgrounds of top and bottom screen are initially white, but can set a background to them
- Background music can be played from downloaded music (mp3, wav, etc.)
    - Users can created albums inside
- Sound effects for opening specific pages and menus (Select, Details, Menu buttons) can be set
- Background wallpapers can be changed on either screen
- Fluid transitions and responsive interactions
- Emulators
    - NES/SNES/GB/GBC/GBA/N64/PS1/Genesis/Sega CD/Saturn: RetroArch
    - Gamecube/Wii: Dolphin
    - Wii U: Cemu
    - DS: melonDS
    - 3DS: Azahar
    - Switch: Eden
    - PS2: NethersX2
    - PS3: aPS3e
    - PSP: PPSSPP
    - PS Vita: EmuCoreV
    - Dreamcast: Flycast

2. User Interface & Hardware Mapping

Use the https://fonts.google.com/specimen/Nunito?query=Nunito font, by defining it in Type.kt

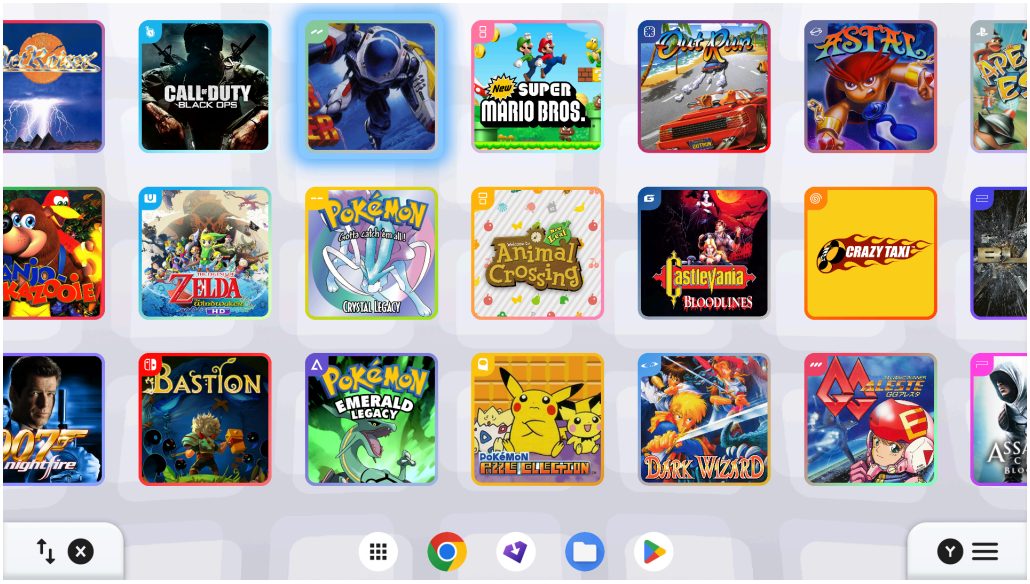
Top Screen

Calico Launcher

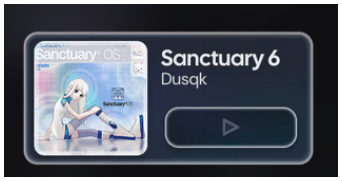
- **Center** of the screen includes splash art (likely 9x16~), with the outlines of the art in a Feathered Edge, a Soft Mask, an Alpha Gradient Mask
- **Upper left** of the screen allows for connections to Discord and to message
- **Upper right** of the screen includes time (military time), date (month/day), & battery percentage (with a battery icon, indicating battery status) (this information is separated by a '|')
    - White, rectangular background to hold this information
    - Top left and bottom left of the rectangular background has some border radius
    - Some padding above this section
- **Bottom right** of the screen includes general menu buttons
    - White, rectangular background to hold this information
    - Top left and bottom left of the rectangular background has some border radius
    - Some padding below this section
    - Include button icons for:
        - A (Select)
        - B (Back)
        - - (Details)
            - Game Description
            - Release date, developer, genre
            - Hours played
            - See when it was last played and play session
        - + (Menu)
            - Add game to favorites
            - Change which emulator to run the game
            - Choose Emulation folder
                - Must have ROMS, Media, & metadata.db enclosed
            - Controller / Input mapping

Calico Launcher

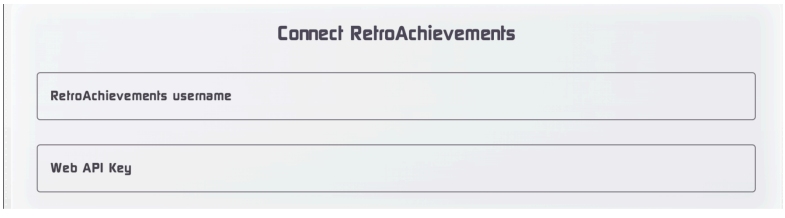
Bottom Screen



- Interactive game grid that shows the games which console they are from
    - These borders and assets are already provided
- When selected, games have a light blue box shadow
- Folders can be created by holding an icon on the screen
    - The icons will start wiggling, and icons can be dragged and dropped
    - Folders can be named
    - Very similar to iOS
- Includes a song section, designed for music (YouTube, Spotify)
    - Opens in web browser of choice

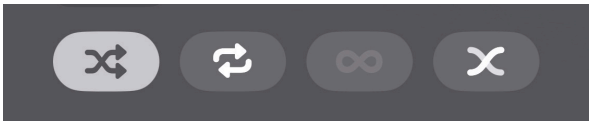


- **Bottom middle** of screen includes taskbar (similar to Windows) that shows android apps
    - Also shows achievements icon (trophy) for each game (using RetroAchievements)



- **Bottom left** of screen includes a sorting / filtering function, where games can be sorted by console, name, last played, total hours spent, & favorites (X button)

    - Use the sorting icon
    - Hamburger menu pops from the left side of the screen, the rest of the screen dims a bit
- **Bottom right** of screen includes the menu function (Y button)
    - Use the 3 lines icon
    - Hamburger menu pops from the right side of the screen, the rest of the screen dims a bit
    - The top shows the current background music
        - Allows skipping, looping, moving to a specific timestamp, moving forward, moving backward, selecting a different song, selecting an album, favoriting a song, shuffling
        - 

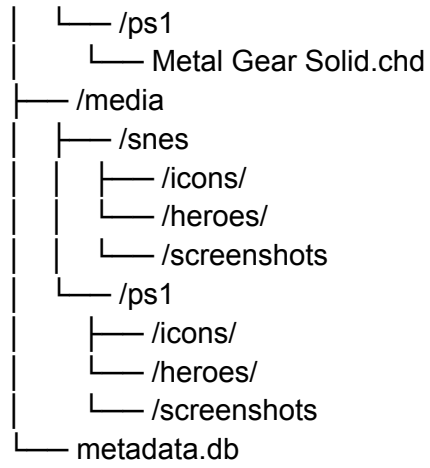
        - 
    - Can create bookmarks of favorite websites and add icons of choice
        - E.g. manga, shows
    - Create song albums and add icons of choice
    - Modify the taskbar
    - Modify the wallpapers
    - Manually change icon / hero / metadata
    - Login to ScreenScraper / SteamGridDB
    - L to cycle to the next left game
    - R to cycle to the next right game
    - ZL to search for a game
    - ZR to move down a full screen of games

3. Application Architecture

File Structure

```
/Emulation
├── /roms
│   ├── /snes
│   │   ├── Chrono Trigger.sfc
│   │   └── Super Metroid.sfc
```

Calico Launcher



Technical Aspects

Data Storage: SQLite

Frontend: Kotlin + Android Studio

- Reads metadata.db and populates information and art
- If a new rom is added, call the APIs
    - Platform folder: /roms/snes, /roms/ps1, etc.
    - File extension: .sfc, .gba, .chd, .iso, etc.
    - File hash: CRC32 / MD5 / SHA1.
    - ScreenScraper / DAT lookup.
    - Filename fallback.
    - Manual user correction.
- Intents hardcoded per emulator in Kotlin
- Intents
    - Platform
    - Emulator
    - Package name
    - Required file types
    - Launch method
    - content_uri or file_path
    - Tested on device
    - Known limitations
- The database stores emulator identity, platform support, and user preferences only
- Create classes for hardcoding launch
    - RetroArchLauncher
    - DolphinLauncher
    - CemuLauncher
    - MelonDsLauncher

Calico Launcher

    - AzaharLauncher
    - EdenLauncher
    - NetherSx2Launcher
    - Aps3eLauncher
    - PpssppLauncher
    - EmuCoreVLauncher
    - FlycastLauncher

Backend: Kotlin

- Call ScreenScraper to fill the the tables for Games, Publishers, Developers, Genre, and maps
    - Called to gather screenshots and put them in the screenshots folder
- Call SteamGridDB to fill heroes

1. Broaden the API Request

- **Request:** `GET /api/v2/heroes/game/12345?styles=alternate,material&types=static`
- **Result:** The API returns a JSON array of the top 10 or 20 background images for that game, regardless of their exact pixel count. Crucially, this JSON payload includes the `width` and `height` integers for every image.

2. Implement an Aspect Ratio Algorithm

1. Read the `width` and `height` of the current JSON object.
2. Divide `width` by `height` (`1920 / 1080 = 1.777...`).
3. Write a condition that accepts an image if the ratio falls within an acceptable floating-point tolerance (e.g., between `1.76` and `1.78`). This catches all the "fuzzy" uploads that are virtually 16:9.

3. Set a Minimum Resolution Threshold

An image could be a perfect 16:9 ratio but only be `400x225` pixels, which would look terrible.

- Ensure the `width` is greater than or equal to a minimum acceptable baseline (e.g., `width >= 1280`).

4. The Selection Protocol

The script processes the array from top to bottom (which is automatically sorted by community upvotes from the API). The very first image in the array that passes both the Aspect Ratio test and the Minimum Resolution test is the winner.

4. Database

Calico Launcher

- Game_id's may not be present in some of the maps - no need to show information in that case
- Game_desc may not be present as well

tablename: **games**			
attribute name	attribute data type	precision	description
game_id	int	AutoNumber	Each game has a unique id
platform_id	int		FK
sort_title	TEXT		
screenscraper_id	int		Unique
steamgriddb_id	int		Unique
retroachievements_id	int		Unique
last_played_at	TEXT		
game_name	TEXT		Game Name
game_desc	TEXT		Synopsis section from ScreenScraper in English (or another language if it's not English exclusive)
release_date	TEXT		
manual_metadata	TEXT		

Calico Launcher

duration_seconds	int		Tracked
is_favorite	bool		

tablename: **game_files**			
attribute name	attribute data type	precision	description
file_id	int	AutoNumber	PK
game_id	int		FK
platform_id	int		FK
file_type	TEXT		base disc playlist update dlc patch mod texture_pack generated unknown
file_path	TEXT		
file_name	TEXT		
extension	TEXT		

Calico Launcher

size_bytes	int		
crc32	TEXT		
md5	TEXT		
sha1	TEXT		
region	TEXT		USA, Europe, Japan, World, etc.
version	TEXT		
last_seen_at	TEXT		
added_at	TEXT		
content_uri	TEXT		
disc_number	int		
disc_total	int		
is_primary	bool		the main file Calico launches by default
is_missing	bool		

tablename: **platforms**			
attribute name	attribute data type	precision	description

Calico Launcher

platform_id	int	Autonumber	PK
name	TEXT		
rom_folder_name	TEXT		
supported_extensions	JSON		
default_emulator_id	int		

tablename: **emulators**

attribute name	attribute data type	precision	description
emulator_id	int	Autonumber	FK
display_name	TEXT		
package_name	TEXT		
install_url	TEXT		
is_enabled	bool		

tablename: **emulator_platforms**

attribute name	attribute data type	precision	description

Calico Launcher

emulator_platform_id	int	Autonumber	PK
platform_id	int		FK
emulator_id	int		FK
supports_content_uri	bool		
supports_file_uri	bool		
direct_launch_supported	bool		For reference if the emulation can't launch
requires_manual_import	bool		

tablename: **game_emulator_preferences**			
attribute name	attribute data type	precision	description
game_id	int		FK For changing which emulator to run the game
emulator_id	int		FK
platform_id	int		FK
is_default	bool		

Calico Launcher

tablename: **collections**			
attribute name	attribute data type	precision	description
collection_id	int	Autonumber	PK
name	TEXT		
icon_path	TEXT		
sort_order	int		
created_at	TEXT		

tablename: **collection_games**			
attribute name	attribute data type	precision	description
collection_id	int		FK, composite PK w/ game_id
game_id	int		FK, composite PK w/ game_id
sort_order	int		Where the game appears in the collection

tablename: **play_sessions**			
attribute name	attribute data type	precision	description
session_id	int	Autonumber	PK

Calico Launcher

game_id	int		FK
emulator_id	int		FK
started_at	TEXT		
ended_at	TEXT		
duration_seconds	int		

tablename: **scan_roots**			
attribute name	attribute data type	precision	description
root_id	int	Autonumber	Choosing the emulation folder
root_path	TEXT		
roms_path	TEXT		
media_path	TEXT		
db_path	TEXT		
last_scanned_at	TEXT		

tablename: **scan_events**			
attribute name	attribute data type	precision	description
scan_id	int	Autonumber	Debugging, PK

Calico Launcher

started_at	TEXT		
ended_at	TEXT		
status	TEXT		
message	TEXT		

tablename: **settings**			
attribute name	attribute data type	precision	description
key	TEXT		Good for current wallpaper, theme, last selected game, music volume, etc. PK
value	TEXT		

tablename: **taskbar_items**			
attribute name	attribute data type	precision	description
item_id	int	Autonumber	PK
name	TEXT		
package_name	TEXT		
icon_path	TEXT		

Calico Launcher

item_type	TEXT		app, bookmark, system_action
sort_order	int		
is_enabled	bool		

tablename: **music_tracks**			
attribute name	attribute data type	precision	description
track_id	int	Autonumber	PK
title	TEXT		
artist	TEXT		Probably optional since these are audio files
file_path	TEXT		
content_uri	TEXT		
duration_seconds	int		
is_favorite	bool		

tablename: **music_albums**			
attribute name	attribute data type	precision	description
album_id	int	Autonumber	PK

Calico Launcher

name	TEXT		
icon_path	TEXT		
sort_order	int		Order in UI (can be changed)

tablename: **music_albums_tracks**			
attribute name	attribute data type	precision	description
album_id	int		FK, composite PK w/ track_id
track_id	int		FK, composite PK w/ album_id
sort_order	int		Song order in UI (can be changed)

tablename: **web_bookmarks**			
attribute name	attribute data type	precision	description
bookmark_id	int	Autonumber	PK
title	TEXT		
url	TEXT		
icon_path	TEXT		

Calico Launcher

sort_order	int		Order in UI (can be changed)

tablename: **input_bindings**			
attribute name	attribute data type	precision	description
binding_id	int	Autonumber	PK
action	TEXT		select, back, details, menu, cycle_left, cycle_right, search, etc.
key_code	int		Android code for the physical button/key that triggers an action
controller_name	TEXT		May have different controller types

tablename: **publishers**			
attribute name	attribute data type	precision	description
publisher_id	int	AutoNumber	PK
publisher_name	TEXT		

tablename: **developers**			
attribute name	attribute data type	precision	description

Calico Launcher

developer_id	int	AutoNumber	PK
developer_name	TEXT		

tablename: **genres**			
attribute name	attribute data type	precision	description
genre_id	int	AutoNumber	Each genre has a unique id, PK
genre_name	TEXT		

tablename: **game_publishers_map**			
attribute name	attribute data type	precision	description
game_id	int		FK
publisher_id	int		FK

tablename: **game_developers_map**			
attribute name	attribute data type	precision	description
game_id	int		FK
developer_id	int		FK

Calico Launcher

tablename: **game_genres_map**			
attribute name	attribute data type	precision	description
game_id	int		FK
genre_id	int		FK

tablename: **assets**			
attribute name	attribute data type	precision	description
asset_id	int	Autonumber	PK
asset_scope	TEXT		Game, ui, collection, taskbar, music_album, bookmark
asset_type	TEXT		Screenshot, hero, icon, boxart, logo, banner, video, top_wallpaper, bottom_wallpaper, menu_sound, back_sound, folder_icon, album_icon, bookmark_icon
game_id	int		FK, nullable
file_name	TEXT		
type	TEXT		type: screenshot, hero, icon
provider	TEXT		ScreenScraper, SteamGridDB, or Manual

Calico Launcher

local_path	TEXT		
source_url	TEXT		
width	int		
height	int		
aspect_ratio	TEXT		
is_selected	bool		
sort_order	int		
content_uri	TEXT		

5. Testing

1. Replicating the Dual Screens (The AVD)

Open the Device Manager in Android Studio and click "Create Device."

Click New Hardware Profile and manually enter the exact screen resolutions and physical dimensions 1080x1920 (16:9) AND 1080x1240

When testing your Jetpack Compose UI, use Android's WindowMetricsCalculator to detect the hinge/fold, ensuring your "Top Screen" splash art and "Bottom Screen" grid render in their respective halves.

2. Mocking the Controller Inputs

Your design document specifies physical button mappings (L/R, ZL/ZR, X, Y). You cannot test these properly with a mouse click.

Calico Launcher

Android Studio’s emulator supports direct gamepad passthrough:

Connect a standard Bluetooth controller and launch AVD.

Android Studio will automatically map your physical controller to the emulator's Android OS.

In your Kotlin code, implement onKeyDown and onKeyUp listeners (or Compose's Modifier.onKeyEvent) to capture the KeyEvent.KEYCODE_BUTTON_X, KEYCODE_BUTTON_Y, KEYCODE_BUTTON_L1, etc. You can now physically press your Xbox controller to test if your UI cycles left and right.

3. Testing the Emulator Intents (The Handoff)

You cannot easily install DuckStation or RetroArch on an x86 PC emulator to test if your game actually boots. However, you can test if your Intent is formatted correctly.

In Android Studio, check the Logcat window. When you select a game in your UI and trigger the Intent, log the exact URI and ComponentName you are trying to launch:
Kotlin

```
Log.d("LauncherIntent", "Target: ${intent.component?.packageName}")
Log.d("LauncherIntent", "Payload: ${intent.getStringExtra("ROM")}")
```

If the Logcat prints out the correct package name and the correct local file path to your dummy .sfc file, your frontend logic is flawless. When you eventually run that exact same APK on the real Ayn Thor hardware, it will seamlessly hand off the file to the real emulator.

4. Testing files

An SD Card with that folder structure will be used to test.